

POSITIONING OF EMERGENCY FACILITIES IN AN OBSTRUCTED TRAFFIC GRID

TEAM: JEFF CRONK, DUFF HOWELL, KEITH SAIN
FACULTY ADVISOR: H. A. KRIEGER

Department of Mathematics
Harvey Mudd College
Claremont, CA 91711

Abstract—We developed algorithms to determine the optimal location of emergency service centers in a given city. We give consideration both to the peculiarities of a particular town, Rio Rancho, and to the procedure by which problems of this nature can be solved in general. The streets of Rio Rancho are laid out in a uniform grid, except for emergency service at each block in the grid. We are given data for the demand transit times between any two points on the grid for Rio Rancho. To do this, we calculated a distance on a grid with no obstacles, then adjusted for the particular obstructions. We first assumed that demand was concentrated at the center of each block and that stations were located at street intersections. Under these assumptions, the problem is amenable to an exhaustive trial of all possibilities. We then made the more sophisticated assumption that the distribution of demand is continuous, and that the emergency facilities could be located anywhere on the streets. We prove two results which greatly reduce this problem: (1) it is necessary that the location of such emergency station be at an intersection; and (2) it is an equivalent problem to consider the demand to be located discretely at the center of each block, rather than being continuously distributed over the block. Due to these results, it is possible to run an exhaustive case study in this case as well. Statistical distributions were calculated to provide a means of comparison of the optimal result with the average result. We studied the stability of the it to be very stable in the first case, and reasonably stable in the latter. Finally, we discuss how similar problems would be approached, and in particular, how an approximation scheme could be implemented if the large number of possibilities prohibits an exhaustive analysis.

RESTATEMENT OF THE PROBLEM

The township of Rio Rancho has hitherto not had its own emergency facilities. It has secured funds to erect two emergency facilities in 1986, each of which will combine ambulance, fire, and police facilities. Figure 1 indicates the demand, or number of emergencies per square block for 1985. The "L" region in the north is an obstacle, while the rectangle to the south is a park with a shallow pond. It takes an emergency vehicle an average of 15 sec to go one block in the N-S direction and 20 sec in the E-W direction. Our task is to locate the two facilities so as to minimize the total response time.

GENERAL ASSUMPTIONS

We modelled an urban traffic grid with two obstructions. We made the following assumptions:

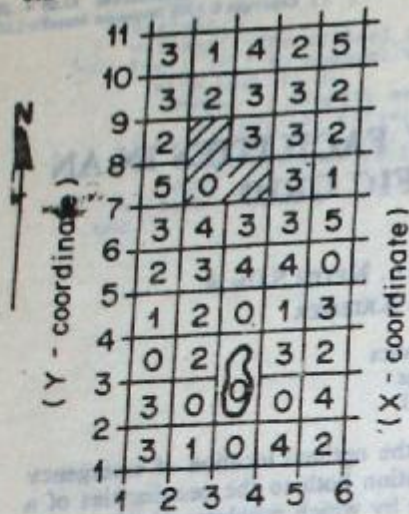


Fig. 1. Demand distribution grid.

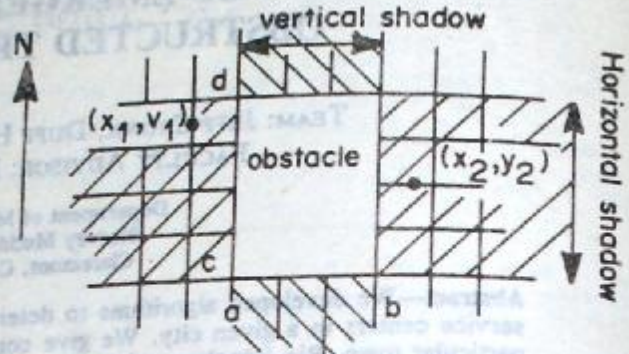


Fig. 2. Rectangular obstruction.

1. Our first model assumes that the demand for service is concentrated at the center of each block and that the facilities are located at intersections.
2. Our second model assumes that demand for emergency service is uniformly distributed on the streets bordering each block and that the facilities may be located anywhere on a street.
3. No service will ever be needed inside any of the obstructions.
4. The number of emergencies per year is small enough that no two problems will occur simultaneously.
5. Intersections and turning require negligible transit time. Only travel along streets takes significant amounts of time.
6. No particular route between two points is preferable over another that takes an equal amount of time to traverse.
7. The future distribution of demands for service will not shift significantly relative to the present distribution.
8. Both emergency facilities have equal capability and effectiveness in handling any emergency. There is no reason to prefer one over the other.

ANALYSIS AND MODEL DESIGN

In order to minimize the average response time of the emergency vehicles, we must have a method of finding the transit time between any two points on the grid. Let $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ be the E-W and N-S coordinates of two points on the grid. In general, the transit time between P_1 and P_2 is just the sum of the horizontal and vertical displacements. However, if both points are in the same column, that is, if the integer parts $[x_1]$ and $[x_2]$ of their x -coordinates are equal, then one must account for the extra time spent driving E-W across the column before heading N-S and then driving W-E to get to the second point. A similar result holds for rows. One adds in the minimum side to side movement. Total distance travelled is then multiplied by time per block to give net travel time. Symbolically, we have

(i) P_1 and P_2 not in same row or column:

$$t = (\text{E-W time}) * |x_1 - x_2| + (\text{N-S time}) * |y_1 - y_2|.$$

(ii) P_1 and P_2 in same row ($[y_1] = [y_2]$):

$$t = (E-W \text{ time}) * |x_1 - x_2| + (N-S \text{ time}) * \min((y_1 + y_2 - 2 * [y_1]), (2 * [y_1 + 1] - y_1 - y_2)).$$

(iii) P_1 and P_2 in same column ($[x_1] = [x_2]$):

$$t = (N-S \text{ time}) * |y_1 - y_2| + (E-W \text{ time}) * \min((x_1 + x_2 - 2 * [x_1]), (2 * [x_1 + 1] - x_1 - x_2)).$$

Note that (ii) and (iii) reduce to (i) when the coordinates are all integers.

This calculation must be corrected for any obstacles in the grid, because it will then be necessary in some cases for an emergency vehicle to travel an additional distance. We first consider a rectangularly shaped obstruction.

By inspection, one sees that a correction to the original distance measurement with no barrier is necessary only if the points are on opposite sides of the obstruction within either the vertical or horizontal shadow shown. An obstruction must have a width of at least two blocks to cast a shadow since a one block barrier is just a block, not an obstruction.

Points (x_1, y_1) and (x_2, y_2) are in the horizontal shadow of the obstruction shown in Fig. 2. The E-W transit time is unchanged by the obstruction. The N-S correction will be the minimum of the vertical detours going either north or south around the obstruction, minus the initially calculated N-S travel time. In terms of the variables pictured:

$$\text{correction} = (N-S \text{ time}) * (\min(2 * d - y_1 - y_2, y_1 + y_2 - 2 * c) - (\text{no obstacle N-S distance traveled})).$$

A similar result holds for travel that includes an East-West detour.

Next we consider an algorithm for computing the correction for an L-shaped barrier, as we have in the town of Rio Rancho. If one of the points is not on the small L-shaped perimeter strip as shown in Fig. 3, then the correction may be calculated using the algorithm for rectangular obstructions.

If one point lies on the small-L strip, then a correction is needed if the other point is anywhere in the shaded area shown. The correction is complicated by the differing traffic flow rates in the N-S and E-W directions.

The general algorithm essentially walks around the L in both directions, takes the

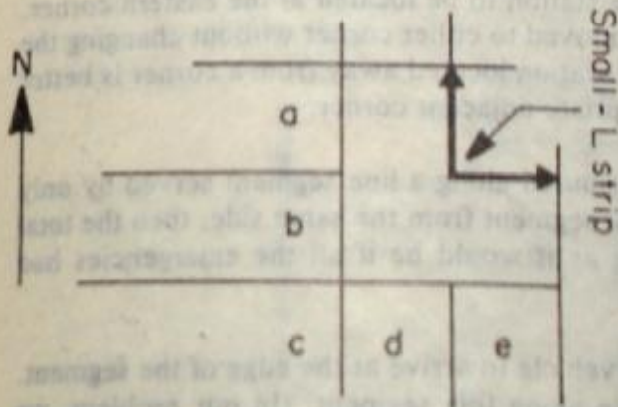


Fig. 3. L-shaped obstruction.

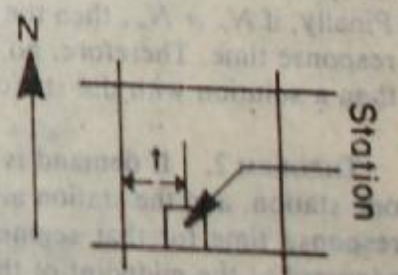


Fig. 4. Non ideal station location.

quicker of the two paths, and subtracts off any appropriate parts of the original time measurement so as to avoid double counting. The correction algorithm must consider different cases for five separate regions, labeled a, . . . , e in Fig. 3, for one point on the vertical part of the small-L, and a similar set of regions for a point on the horizontal. The cases are tested and corrected for by the function L-shape-correction in the program listing (see the Appendix).

Model 1

In our first model, we assume that the demand for emergency services is located at the center of each block, and that the emergency facilities must be built at an intersection. There are 66 intersections on the grid, which means that there are $66 \times 65 = 4110$ possible locations for the two emergency centers. There are 50 blocks and thus 50 possible locations for an emergency. It is then just a matter of trying each of these possibilities, and minimizing the response time.

Model 2

We now extend the first model by considering the demand for emergency service to occur uniformly along the streets of each block. We also assume that we may build an emergency center anywhere along the streets of the town. However, we can immediately prove two results that simplify the problem, and return us to a discrete problem.

THEOREM 1. If an emergency facility is not located on a corner, then the net response time can be reduced by moving it to an appropriate corner.

Proof. Consider a station located somewhere not at an intersection as shown in Fig. 4.

Every time an emergency vehicle leaves this station, it must begin by travelling either east or west. Let N_e be the number of emergencies per year that the vehicle approaches by travelling east, and N_w be the number approached travelling west. Suppose N_e and N_w are unequal, say $N_e < N_w$, and let t be the time it takes for the vehicle to travel from the station to the western corner.

Now consider relocating the emergency station to the western corner. In one year, there will be N_e emergencies for which it will take t longer to respond to, but there will be even more N_w emergencies for which the response time is shortened by t . Thus, moving to the western corner reduces the total response time, producing a better solution. Similarly, if $N_e > N_w$, it would be better for the station to be located at the eastern corner. Finally, if $N_e = N_w$, then the station can be moved to either corner without changing the response time. Therefore, no solution with a station located away from a corner is better than a solution with the station on an appropriate adjacent corner.

THEOREM 2. If demand is uniformly distributed along a line segment served by only one station, and the station always enters the segment from the same side, then the total response time for that segment is the same as it would be if all the emergencies had occurred at the midpoint of the segment.

Proof. Let t_0 be the time it takes for the vehicle to arrive at the edge of the segment. Let v be the speed of the emergency vehicle along this segment. (In our problem, we

have $v = 1$ block/15 sec for N-S streets and $v = 1$ block/20 sec for E-W streets.) The number of emergencies that occur in one year in a subsegment of width dx is proportional to dx . Let N be the proportionality constant. The time it takes the vehicle to respond to an emergency a distance x from the nearest corner is $t_0 + x/v$. Thus, the total time T spent responding to emergencies on the whole segment is the time it takes to reach each subsegment multiplied by the number of emergencies on that subsegment per year, integrated over all subsegments. Thus

$$\begin{aligned} T &= \int_0^L \left(t_0 + \frac{x}{v} \right) (N dx) \\ &= N (t_0 x + x^2/2v) \Big|_0^L \\ &= NL (t_0 + L/2v) \end{aligned}$$

Now the number of emergencies occurring in one year on the entire segment is NL ; the time it takes for the emergency vehicle to arrive at the midpoint is $(t_0 + L/2v)$. Thus, T is the same as it would be if all emergencies had occurred at the midpoint of the segment.

Intuitively speaking, for every emergency occurring further than the midpoint, there is another which is nearer by the same amount, so the average arrival time is just the time it takes to get to the midpoint of the segment.

On this basis, we can turn the continuous distribution into a completely equivalent discrete problem by considering a quarter of the emergencies on each block to occur at the midpoints of each of the four streets surrounding the block.

We will have to make a correction in one special case. Consider when there is a street segment for which part of the block is closest to one station and the rest is closer to the other. In this case, we cannot replace the continuous distribution by a single point at which all emergencies occur. Split the street into two segments, divided by the point equidistant from the two stations, as in Fig. 5.

We must divide up the demand for service into two parts, A and B, with $A + B =$ total demand and $A/B = (\text{segment A}/\text{segment B})$. The demands are located at the center of the segments.

These results mean that we only have to consider again the 4110 intersections as locations for the emergency stations, and the 112 segments of streets as possible sites for an emergency. This is easily done by direct calculation.

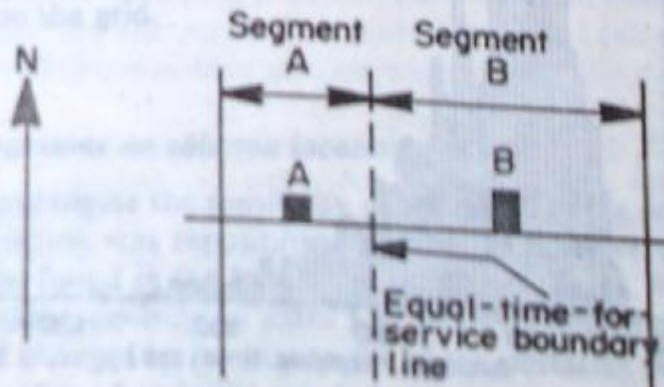


Fig. 5. Special demand distribution for divided service streets.

RESULTS AND INTERPRETATIONS

Optimal solutions

A computer simulation based on our theorems and algorithms was run using the obstacles and demand distribution specified in the problem statement. Optimum solutions were obtained for the discrete and continuous distribution cases. For Model 1, the five best configurations are as follows:

P_1	P_2	Average response time (sec)
(4, 5)	(4, 9)	47.0
(4, 5)	(4, 10)	47.5
(4, 5)	(3, 9)	47.6
(3, 5)	(4, 9)	47.7
(4, 5)	(3, 10)	47.7

The full distribution of configurations is graphed in Fig. 6. The average distribution has a response time of 66.0 sec and the standard deviation is about 12.2 sec. For Model 2, the five best configurations are

P_1	P_2	Average response time (sec)
(4, 5)	(4, 9)	47.0
(4, 4)	(4, 9)	47.0
(4, 5)	(4, 10)	47.1
(3, 5)	(4, 9)	47.2
(4, 5)	(3, 9)	47.2

This case has a distribution with mean response time of 65.7 sec with a standard deviation of 12.1 sec. (See Fig. 7.)

This shows that the advantage of a careful selection of station positions over a random selection is about 18 sec/emergency, a 28% decrease in average response time.

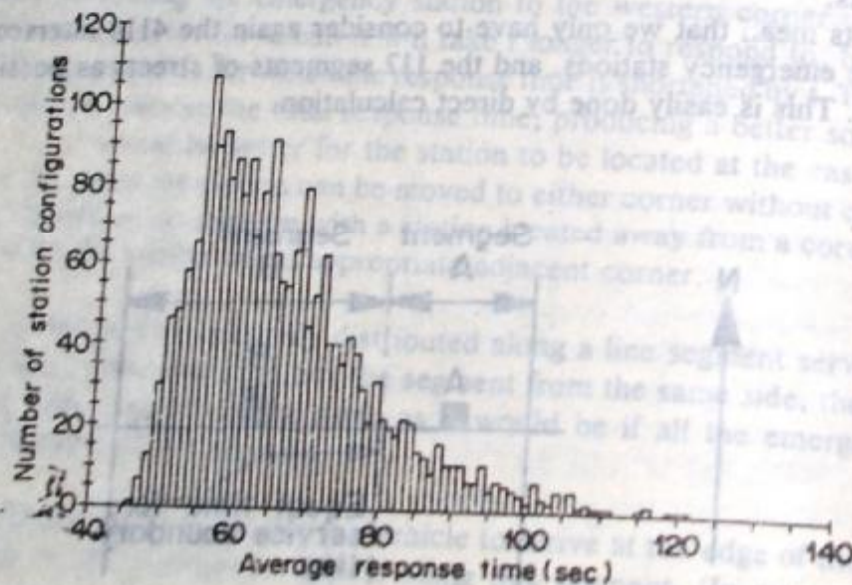


Fig. 6. Distribution of response time versus number density of the state for the discrete model.

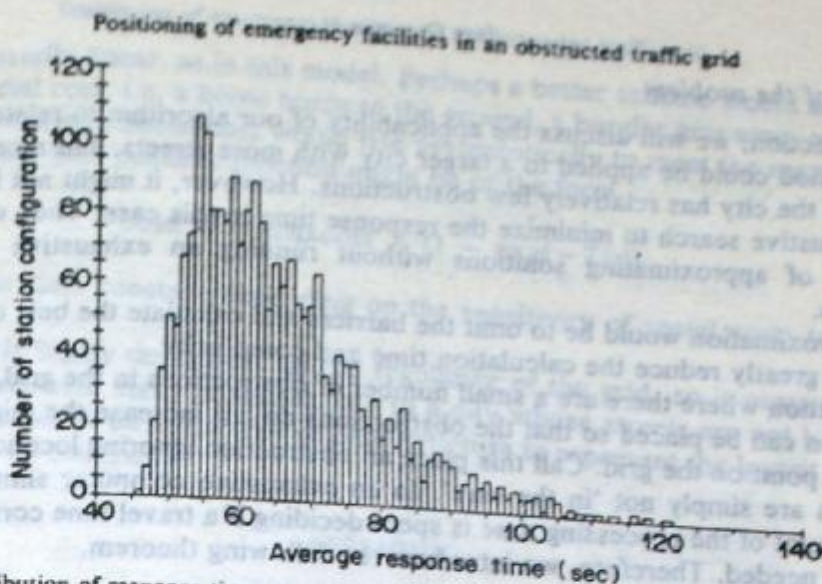


Fig. 7. Distribution of response time versus number density of the state for the continuous model.

Stability of the solution points

We must ask how good our solution is if we remove the assumption that the demand distribution is constant over time. If it is assumed that demand changes in a random fashion over time and that the number of such changes is large, then the long-range average distribution is flat, with an equal number of calls from any block. We set the demand equal to 1 for all city blocks, except for obstacles, which retain their demand of zero. In this case, the optimal statement coordinates were

$$(4, 4) (4, 9).$$

Our data-specific coordinates of (4, 5) (4, 9) were only 1.96 sec slower than this configuration. Thus, for the case of a highly volatile demand function, the proposed solution coordinates are almost as good as the uniform demand function in the long run.

As another extreme test, the original demand distribution was inverted, swapping high for low— $D = 5 - D$. In this case, the original optimization coordinates are 5.14 sec slower. However, this is still an above-average solution.

The reason that the optimal solutions work well under different distribution data is that unless the distribution is extremely lopsided, any optimal solution will have quick access to most points on the grid.

Effects of obstructions on solution location

In order to investigate the sensitivity of the model to the position of the obstacles, the L-shaped obstruction was repositioned so that the inner vertex of the L coincided with the (4, 9) station found in the best solution. The (4, 5), (4, 9) configuration became the 104th best solution, requiring an extra 3.79 sec per event to respond. All in all, we found that even small changes in the placement of the obstacles can lead to significant degradation of the quality of an initial solution.

Extension of the problem

In this section, we will discuss the applicability of our algorithm to related problems. Our method could be applied to a larger city with more streets, and more emergency facilities, if the city has relatively few obstructions. However, it might not be feasible to do an exhaustive search to minimize the response time in this case. Thus we must have some way of approximating solutions without running an exhaustive study of all possibilities.

One approximation would be to omit the barriers and calculate the best configuration. This would greatly reduce the calculation time and complexity.

In a situation where there are a small number of obstructions in the grid, it is possible that a station can be placed so that the obstructions do not increase the minimum transit time to any point on the grid. Call this place an obstruction ignoring location (OIL). The obstructions are simply not 'in the way'. In an exhaustive computer simulation, a significant amount of the processing time is spent deciding if a travel time correction due to obstacles is needed. Therefore, we introduce the following theorem.

THEOREM 3. If the optimum station position is solved for on a grid with obstructions ignored, and the ideal station positions are OILs, then the solution is the optimum solution for the grid with obstructions taken into consideration.

Proof. (1) The no-obstruction solution minimizes the net response time (NRT) to emergencies over a period of time. (2) Obstacles can only increase the NRT, or leave it unchanged; they can never decrease it. If the emergency stations are both OILs, then the NRT is unchanged due to obstacles, by definition of an OIL. For all other stations, the initial NRT is greater than or equal to the best NRT, and, by (2), their obstructed NRT is greater than or equal to their unobstructed NRT. Therefore

$$\text{NRT}(\text{optimal}) \leq \text{NRT}(\text{others, unblocked}) \leq \text{NRT}(\text{others, blocked})$$

Thus, the unobstructed optimal solution is the obstructed optimal solution if it is an OIL.

This result is a general result for traffic grids and is independent of the distribution of demand.

When Model 1 is solved with the obstacles removed, the best configuration is (4, 5), (4, 9), just as it is with the obstacles present. It turns out that (4, 5) and (4, 9) are OILs so that we could have known the solution to Model 1 without ever considering the obstacles!

In general, however, the no-obstacle solution will not necessarily be an OIL, and then we cannot know beforehand if the solution will be a good one when the obstacles are present. In fact, when model 2 is solved without obstacles, the best configuration is (4, 5) and (3, 9), but when the obstacles are present, the next-best solution (4, 5) and (4, 9) becomes the best solution.

Another way to produce an approximate solution for a large problem is to consider a subgrid small enough to try all possibilities. After this approximation, successive trials would involve configurations near to that which was optimal on the subgrid.

We would be able to treat the case where the demand function varies continuously but not linearly throughout the streets. We would not then be able to use the short-cut of assuming the demand to be concentrated at single points, but a scheme could be devised to approximate response times by integrating over city blocks.

Finally, we can also consider the social costs of emergencies as a general function of

time, not necessarily linear, as in this model. Perhaps a better scheme would be to assign a maximum social cost, i.e. a home burns to the ground, a burglar gets away with everything, patients die, etc. Social cost would rise asymptotically to meet the maximum cost as time increased. One possible function might be of the form

$$\text{cost}(T) = \text{maxcost} \times (1 - \exp(-T/\tau))$$

where τ is some time constant depending on the sensitivity of social costs to response time.

Our method is highly dependent on the regularity of the grid, so it cannot be easily extended to a city with many obstacles, nor to a city whose streets are not laid out in a grid. For these cases, a different type of data structure to represent the layout of the city would probably be necessary.

APPENDIX

```

Program Optimize (input, output, infile, outfile, distfile);
( Developed 2/7-10/86 for the Mathematics Competition in Modeling )
( Jeff Cronk )
( Duff Howell )
( Keith Saints )

( Problem B. )
( This program finds the optimal location for emergency )
( service centers for the town of Rio Rancho. )

CONST
  NS_speed = 15; ( Number of sec. to go 1 block N-S )
  EW_speed = 20; ( Number of sec. to go 1 block E-W )
  time_interval = 1; ( For distribution statistics )
  number_of_solutions = 5; ( Number of solutions to keep )
  Worst_Avg_Response_Time = 150; ( The worst possible solution )

  number_of_intervals = Worst_Avg_Response_Time div time_interval;

  X_max = 6;
  Y_max = 11; ( Dimensions of the city )

  Station1_xmin = 1;
  Station1_xmax = 6;
  Station1_ymin = 1;
  Station1_ymax = 11; ( Search range for Station 1 )

  Station2_xmin = 1;
  Station2_xmax = 6;
  Station2_ymin = 1;
  Station2_ymax = 11; ( Search range for Station 2 )

  Obstructions = True; ( Account for obstructions? )

  Events_Distributed = True; ( If True, events are considered to )
  ( at locations evenly distri- )
  ( buted along each block. )
  ( If False, events are considered )
  ( to occur only at the center of )
  ( the block. )

  pondmin_x = 3; ( these four constants describe the pond )
  pondmin_y = 2; ( they are used to compute the distance )

  pondmax_x = 4; ( correction factor. )
  pondmax_y = 4;

  ellmin_x = 2; ( these four constants describe the "L" )
  ellmax_x = 4; ( shaped obstruction. They are used to )

```

```

elimin_y = 7;      { compute the correction factor for the }
elimax_y = 9;      { "L" shaped obstruction.           }

TYPE
  stationtype = Record
    x, y,
    ResponseTime: Real;
  End;

  solutiontype = Array [1..number_of_solutions] of
    Record
      station1,
      station2: StationType; { The two stations }
      Time: Real;             { Average Response Time }
    End;

VAR
  Counter,
  X_Block, Y_Block      : integer;
  infile, outfile,
  distfile              : text;
  BestSolutions         : SolutionType;
  Number_of_Emergencies,
  Events_per_block      : real;
  Events_per_year       : Array [0..2*X_max, 0..2*Y_max] of real;
  Distribution          : Array [0..number_of_intervals] of integer;

Function Min2(a,b:Real):Real;
Begin
  If a < b
  Then
    Min2 := a
  Else
    Min2 := b
  End;

Function Rectangle_correction ( a1, a2, b1, b2,
                                widelo, widshi, frontlo, fronthi,
                                flow2 :real ):real;
( a1, a2, b1, b2      Are first and second grid point coordinates
  widelo and widshi   Are upper and lower limits of the rectangular
                        obstruction in a direction perpendicular to the
                        street removed by the blockage.
  frontlo & fronthi   Are limits of the blockage in other direction
  flow2               Is the travel times (15 or 20 seconds/block
                        in the 2 direction that we travel extra in )

Var
  temp:real;           { holds value of corection }
  k,needed:boolean;    { whether or not a correction needed }

Begin
  ( If on opposite sides of obstruction and neither above or below it )
  If (a2>widshi) and (a2>widelo) and (b2>widshi) and (b2>widelo)
    and
    ( (a1<=frontlo) and (b1>=fronthi)) or
    ((a1)=fronthi) and (b1<=frontlo) )
  Then
    Begin
      temp:= flow2 * min2( (a2+b2-2*widelo), (2*widshi-a2-b2) );
      If (trunc(a1) <> trunc(b1))
      Then
        temp:= temp - flow2*abs(a2-b2)
      Else
        temp:= temp - flow2
          *min2((a2+b2-2*trunc(a2)),
                (2*trunc(a2+1)-a2-b2) );
          { subtracts off original metrics }
          { estimation of travel in l-direction }
      rectangle_correction:=temp
    End
    { of 'if on opposite sides and not above or below' }
  Else
    rectangle_correction:=0;
  End;
  { of function Rectangle_correction }

Function L_correction ( x1, y1, x2, y2,
                        hmin, hmax, vmin, vmax,
                        hflow, vflow :real ):real;
( function calculates correction to distance function if one point is

```

Positioning of emergency facilities in an obstructed traffic grid

621

on the inside, 'small L' perimeter points on the L-shaped obstruction and the other is in a place where the obstruction causes a change in minimum path distance between the points

```

( x1, x2, y1, y2      Positions to measured between
  hmin, hmax          Left and right limits of the L
  vmin, vmax          Highest and lowest bounds of the L
  hflow, vflow        time per block up and across

Begin (of "L" shape correction)
If not ((y2 = vmin+1) and (x2=hmax) and (x2=hmax-1))
Then
  L_correction:=0.0
Else
  If ((x1<=hmin) and (y1<=vmax) and (y1=vmax-1))
  Then
    L_correction:=2*vflow*(vmax-y1)
  Else
    If ((x1<=hmin) and (y1<=vmin+1) and (y1=vmin))
    Then
      L_correction:=min2( (2*vflow), (2*vflow*(y1-vmin) +
        2*hflow*(hmax-x2)) )
    Else
      If ( (x1<=hmin) and (y1<=vmin) )
      Then
        L_correction:=min2( (2*hflow*(hmax-x2)), (2*vflow) )
      Else
        If ((y1<=vmin) and (x1=hmin) and (x1<=hmin+1))
        Then
          L_correction:=min2( (2*vflow + 2*hflow*(x1-hmin)),
            2*hflow*(hmax-x2) )
        Else
          If ((y1<=vmin) and (x1=hmax-1) and (x1=hmax))
          Then
            L_correction:= hflow*( (2*hmax-x1-x2) -
              min2( (x1+x2-2*(hmax-1)), 2*hmax-x1-x2 ) )
          Else
            L_correction:=0;
        End;
      ( of procedure L_correction )

```

Function Distance(x1, y1, x2, y2:real):real;

VAR
Dist : real;

```

Begin
If trunc(x1)=trunc(x2)
Then
  If Trunc(y1)=Trunc(y2)
  Then
    Dist := NS_speed * Abs(y2-y1) + EW_speed * Abs(x2-x1)
  Else
    Dist := NS_speed * Abs(y2-y1)
      + EW_speed * Min2(x1 + x2 - 2*Trunc(x1),
        2*(Trunc(x1) + 1) - x1 - x2)
  Else
    If Trunc(y1)=Trunc(y2)
    Then
      Dist := EW_speed * Abs(x2-x1)
        + NS_speed * Min2(y1 + y2 - 2*Trunc(y1),
          2*(Trunc(y1) + 1) - y1 - y2)
    Else
      Dist := NS_speed*Abs(y2-y1) + EW_speed*Abs(x2-x1);
  (Now correct the distance for the obstructions)

```

If Obstructions

```

Then
  Begin
    (Correct for Pond)
    Dist := Dist + rectangle_correction
      (x1, y1, x2, y2,
        Pondmin_y, Pondmax_y,
        Pondmin_x, Pondmax_x, EW_speed);
    (Correct for the "L" shape as a rectangle)
    (Correct for L East/West)

```

```

Dist := Dist + rectangle_correction
              (x1, y1, x2, y2,
               Ellmin_y, Ellmax_y,
               Ellmin_x, Ellmax_x, EW_speed);
(Correct for L North/South)
Dist := Dist + rectangle_correction
              (y1, x1, y2, x2,
               Ellmin_x, Ellmax_x,
               Ellmin_y, Ellmax_y, NS_speed);

(Correct for the interior points of the L)
Dist := Dist + L_Correction
              (x1, y1, x2, y2,
               Ellmin_x, Ellmax_x, Ellmin_y, Ellmax_y,
               EW_speed, NS_speed);
Dist := Dist + L_Correction
              (x2, y2, x1, y1,
               Ellmin_x, Ellmax_x, Ellmin_y, Ellmax_y,
               EW_speed, NS_speed);
Dist := Dist + L_Correction
              (y1, x1, y2, x2,
               Ellmin_y, Ellmax_y, Ellmin_x, Ellmax_x,
               NS_speed, EW_speed);
Dist := Dist + L_Correction
              (y2, x2, y1, x1,
               Ellmin_y, Ellmax_y, Ellmin_x, Ellmax_x,
               NS_speed, EW_speed);
End;

Distance := Dist;
End; {of Distance function}

Procedure Update_Distribution (time:real);
{ Procedure tallies a total yearly response time in the array
  "Distribution" that forms a distribution of the number of
  solutions that give a average response time for each time interval.}
VAR
  index:integer;
Begin
  index:=Trunc(time/time_interval);
  Distribution [index] := Distribution [index] + 1;
End;

Function WithinRange (x1, y1, x2, y2:real):Boolean;
Begin
  If
    (Station1_xmin <= x1)
    And (Station1_xmax >= x1)
    And (Station1_ymin <= y1)
    And (Station1_ymax >= y1)
    And (Station2_xmin <= x2)
    And (Station2_xmax >= x2)
    And (Station2_ymin <= y2)
    And (Station2_ymax >= y2)
  Then
    WithinRange:= True
  Else
    WithinRange:= False
  End;
End;

Procedure Insert ( Station1, Station2: StationType;
                  VAR Solutions: SolutionType);
VAR
  counter : integer;
  time : real;
  Shift : boolean;
Begin
  time := (station1.responseTime + Station2.responseTime)
    / Number_of_Emergencies;
  counter := number_of_solutions;
  If Time ( Solutions[counter].Time
  Then
    Begin
      Shift:=True;
      Repeat

```

Positioning of emergency facilities in an obstructed traffic grid

623

```

If Time < Solutions[Counter-1].Time
Then
  Begin
    Solutions[Counter]:=Solutions[Counter-1];
    Counter:=Counter-1;
  End
Else
  Shift:=False
Until (counter = 1) or (Shift = False);
Solutions[counter].station1:=station1;
Solutions[counter].station2:=station2;
Solutions[counter].time:=time;
End
End; [of Insert]

Procedure CalculateResponseTime
  (VAR station1: StationType; VAR station2: StationType;
   emergency_x, emergency_y: integer);
VAR
  time1, time2,
  time1A, time2A,
  probl, probl2,
  NewTime1, NewTime2,
  weight, speed
  correction
  : real;
  : Boolean;

Begin
  weight := Events_per_year [emergency_x, emergency_y];
  ( If weight=0 then no changes to the total response times is necessary)
  If weight<>0
  Then
    Begin
      time1 := Distance (Station1.x, Station1.y,
                        emergency_x/2, emergency_y/2);
      time2 := Distance (Station2.x, Station2.y,
                        emergency_x/2, emergency_y/2);

      ( Is the emergency on an N-S street or an E-W street? )
      ( Set speed appropriately. )

      If emergency_x mod 2 = 0
      Then
        speed := NS_speed
      Else
        speed := EW_speed;

      Correction := false;
      (No correction needed)

      ( A special calculation is necessary. Part of the street may
      ( be served by one station and the other part by the other.
      ( If vehicles from both stations enter the street from opposite
      ( direction, then use the normal algorithm to choose the faster. )

      ( Time1A and Time2A are the distances of stations 1 and 2
      ( to a nearby corner. This corner should have the relationship
      ( of a nearer corner with respect to one of the stations and
      ( the farther with the respect to the other if a correction is
      ( needed. If it is the nearer to both, or the farther to both
      ( then no correction is needed.

      If (abs(time1-time2) < speed) and (Events_Distributed)
      Then
        If speed = EW_speed
        Then
          Begin
            Time1A := Distance(station1.x, station1.y,
                              emergency_x/2-1/2, emergency_y/2);
            Time2A := Distance(station1.x, station2.y,
                              emergency_x/2-1/2, emergency_y/2);
            If ( (Time1A < Time1) and (Time2A < Time2) )
              or ( (Time1A > Time1) and (Time2A > Time2) )
            Then
              correction := false
            Else
              correction := true;
            End
          End
        End
      End
    End
  End

```

```

Else
  Begin
    Time1A := Distance(station1.x, station1.y,
      emergency_x/2, emergency_y/2-1/2);
    Time2A := Distance(station2.x, station2.y,
      emergency_x/2, emergency_y/2-1/2);
    If ( (Time1A < Time1) and (Time2A < Time2) )
      or ( (Time1A > Time1) and (Time2A > Time2) )
    Then
      correction := false
    Else
      correction := true;
    End;

    If correction=True
    Then
      Begin
        Probl := (time2-time1+speed) / (2*speed);
        ( The probability that an emergency is closer to #1 )
        Prob2 := 1 - Probl;

        NewTime1 := 1/4 * (3*Time1 + Time2 - Speed);
        NewTime2 := 1/4 * (3*Time2 + Time1 - Speed);
        ( Recalculate the times )

        Station1.Responsetime := Station1.ResponseTime
          + Probl * weight * NewTime1;

        Station2.Responsetime := Station2.ResponseTime
          + Prob2 * weight * NewTime2;

      End
    Else

      If time1 < time2
      Then
        station1.responsetime := station1.responsetime
          + weight * time1
      Else
        If time1 > time2
        Then
          station2.responsetime := station2.responsetime
            + weight * time2
        Else
          ( This case occurs when the two stations are equidistant )
          ( from the emergency, but it fell through the correction )
          ( procedure because the two stations were on the same )
          ( side of the emergency. )
          Begin
            station1.responsetime:=station1.responsetime
              + 0.5 * weight * time1;
            station2.responsetime:=station2.responsetime
              + 0.5 * weight * time2;
          End;
        End;
      End; ( if weight <> 0 )

    End; ( of Procedure CalculateResponseTime )

  Procedure Seek_Optimum;
  ( Find the optimum solution. )

  VAR
    ResponseTime,
    Average_Response_Time :Real;
    counter,
    x1, y1, x2, y2, ( Position of stations )
    emergency_x, emergency_y :Integer; ( Position of events )
    Station1, Station2 :StationType;

  Begin (of procedure Seek_optimum)

  ( Clear the best solutions array)

  For counter := 1 TO number_of_solutions Do
    BestSolutions[counter].time := Worst_Avg_Response_Time;
    (Any solution will be better)

  (Try all the possible combinations of station locations-Only once)

```

```

For y1 := 1 to Y_max Do
  For x1 := 1 to X_max Do
    For y2 := y1 to Y_max Do
      For x2 := 1 to X_max Do
        If (y2 > y1) Or (x2 > x1) Then      [ No Double Counting ]
          If WithinRange (x1,y1,x2,y2) Or WithinRange(x2,y2,x1,y1)
          Then
            Begin
              station1.x := x1;
              station1.y := y1;
              station1.ResponseTime := 0;
              station2.x := x2;
              station2.y := y2;
              Station2.ResponseTime:=0;

              [ Try all possible locations for an emergency ]
              For emergency_x := 2 to 2*X_max Do
                (Divide by 2 to get x-coord. )

                For emergency_y := 2 to 2*Y_max Do
                  (Divide by 2 to get y-coord. )

                  Begin
                    CalculateResponseTime
                      ( Station1, Station2, Emergency_x, Emergency_y );
                  End;

                  Average_Response_Time := (Station1.ResponseTime
                                             + Station2.ResponseTime)
                                             / Number_of_Emergencies;

                  Update_Distribution (Average_Response_Time);

                  Insert (Station1, Station2, BestSolutions);

                End;      [ of stations that are within range ]

              [ This ends the loop in which all possible
                locations for the stations are tried. ]

            End; [ of procedure seek_optimum ]

begin (Main Program)

  [ Set up File for distribution of solutions ]

  Rewrite(distfile);
  For counter:= 0 to number_of_intervals Do
    distribution[counter]:=0;

  [ Read in the number of emergencies per block per year ]
  Reset(infile);

  For Y_Block := 2 to 2*Y_Max do
    For X_Block := 2 to 2*X_max do
      Events_per_year[X_Block, Y_Block]:=0;      [At every half-block]

  Number_of_Emergencies := 0;

  For Y_Block := 1 to Y_Max-1 do
    Begin
      For X_Block := 1 to X_Max-1 Do
        Begin
          read(infile, events_per_block);
          Number_of_Emergencies := Number_of_Emergencies
            + events_per_block;

          If events_distributed
          Then
            Begin

              [ Distribute the number of emergencies among the
                midpoints of the four streets bordering the block ]

              Events_per_year [X_Block*2, Y_Block*2+1] :=
                Events_per_year [X_Block*2, Y_Block*2+1]
                + 1/4 * events_per_block;

              Events_per_year [X_Block*2+1, Y_Block*2+1] :=
                Events_per_year [X_Block*2+1, Y_Block*2+1]
                + 1/4 * events_per_block;
            End;
          End;
        End;
      End;
    End;
  End;

```

```

Events_per_year [X_Block*2+2, Y_Block*2+1] :=
  Events_per_year [X_Block*2+2, Y_Block*2+1]
  + 1/4 * events_per_block;

Events_per_year [X_Block*2+1, Y_Block*2] :=
  Events_per_year [X_Block*2+1, Y_Block*2]
  + 1/4 * events_per_block;

End
Else
  ( All emergencies occur at the middle of the block )
  Events_per_year [X_Block*2+1, Y_Block*2+1]
    := Events_per_Block;

End;
Readln(infile);
End;

Seek_Optimum;

[Output the Best Solutions]
Rewrite(outfile);
Writeln(outfile);
Writeln(outfile, 'Obstructions = ', Obstructions);
Writeln(outfile, 'Events Distributed = ', Events_Distributed);
Writeln(outfile);
Writeln(outfile);
Writeln(outfile,
  Station 1      Station 2      Average Response Time');
Writeln(outfile,
  '-----');
For counter := 1 to Number_Of_Solutions Do
  Begin
    Write(outfile, '(', BestSolutions[counter].station1.x:5:2, ', ');
    Write(outfile, BestSolutions[counter].station1.y:5:2, ', ');
    Write(outfile, '(', BestSolutions[counter].station2.x:5:2, ', ');
    Write(outfile, BestSolutions[counter].station2.y:5:2, ', ');
    Writeln(outfile, ' ', BestSolutions[counter].time:8:3);
  End;

( Write to distribution File )
For counter:=1 to number_of_intervals do
  Writeln(distfile,
    counter*time_interval:7, ' ', distribution[counter]);

End.

```